ECSE 324 Computer Organization

Laboratory 3

Keyboard and VGA

Seyed Yazdan Zinati

April 12, 2021

Abstract

In this laboratory, we explore more complex I/O capabilities of the DE1-SoC by implementing drivers for input/output devices such as the VGA display controller and the PS/2 keyboard port in ARM assembly. We use multiple programs to ensure the correct functionality of implemented I/O drivers as we implement them. Finally, we integrate them into an application capable of switching between rendered drawings on the VGA display using the PS/2 keyboard port.

1 Rendering Images on the VGA Display

To provide a higher-level layer over the primitive functionality offered by the pixel and character buffers, we create drivers to write to the pixel and character buffers to gain easier control to the VGA screen.

1.1 Pixel Buffer Subroutines

We implement two subroutines that interact with the pixel buffer memory. `VGA_draw_point_ASM` inputs three arguments, the first (`R0`) and second (`R1`) arguments are the x and y coordinate of the pixel to which the colour indicated in the third argument (`R2`) is written.

As the pixel buffer provides an image resolution of 320×240, writing the colour argument `R2` into a memory address not corresponding to the pixel buffer memory might cause memory corruption. To prevent errors such as memory corruption, we employ defensive programming by verifying the validity of x and y arguments by checking their bounds ($0 \leq x < 320$ and $0 \leq y < 240$).

If x and y are out of bounds, we return to the caller by branching to the address contained in the Link Register (`LR`). It is noteworthy to mention that ARM Procedure Call Standards are used in implementing driver subroutines, and used registers within subroutines are callee-saved. For instance, register `R3` used to calculate the pixel memory address is pushed onto and popped off the stack at the start and end of the `VGA_draw_point_ASM` subroutine.

If the passed x and y coordinates are within bounds, the corresponding memory address in the pixel buffer memory is calculated by adding the base address of the pixel buffer memory and the logical shift left (`LSL`) results of x by 1 and y by 10. Next, the 16-bit colour is written to the calculated memory address using a half-word store operation (`STRH`).

The `VGA_clear_pixelbuff_ASM` subroutine clears all the valid memory locations in the pixel buffer. It takes no arguments and returns nothing. We clear the pixel buffer by writing the colour black `rgb(0,0,0)` to every memory location in the pixel buffer.

We use the previously implemented `VGA_draw_point_ASM` subroutine to write the colour black to each valid memory address in the pixel buffer. This is done by using a nested loop that loops through x and y within the previously described bounds. The loop counters for x and y are kept track of using `R0` and `R1` respectively and the `VGA_draw_point_ASM` subroutine is called for each valid x and y with `R2` storing `rgb(0,0,0)`. The subroutine branches back to the caller once both x and y reach their upper bounds (319 and 239 respectively).

1.2 Character Buffer Subroutines

The two `VGA_write_char_ASM` and `VGA_clear_charbuff_ASM` subroutines that we implement to interact with the character buffer memory are very similar in their implementation to the earlier defined subroutines. In fact, `VGA_write_char_ASM` and `VGA_clear_charbuff_ASM` subroutines are analogous to the `VGA_draw_point_ASM` and `VGA_clear_pixelbuff_ASM` subroutines accordingly.

`VGA_write_char_ASM` writes the ASCII code passed in the third argument (`R2`) to the screen at x and y coordinates passed in the first two arguments (`R0` and `R1`). As opposed to the pixel buffer resolution of 320×240, the character buffer provides a resolution of 80×60 characters. Thus, we first check the validity of x and y arguments using the following condition: $0 \leq x < 80$ and $0 \leq y < 60$.

Once the two first arguments are validated, we calculate the memory address specified by x and y by adding together the logical shift left (`LSL`) by 7 of y with x and the base address of the character buffer. The ASCII code passed in R2 is then written to the computed memory location using a byte store operation (`STRB`).

To clear all memory location in the character buffer in `VGA_clear_charbuff_ASM`, we follow the same approach as the one described for clearing the pixel buffer. Using a nested loop we loop through all valid x ($0 \leq x < 80$) and y ($0 \leq y < 60$) coordinate of the character buffer and reuse the `VGA_write_char_ASM` subroutine to store 0 in every valid location specified by x and y. The subroutine returns when both x and y hit their upper bounds. x and y loop counters both start at 0.

1.3 Test VGA Driver

To test implemented subroutines for the VGA display, we use the provided code that clears both pixel and character buffers using `VGA_clear_pixelbuff_ASM` and `VGA_clear_charbuff_ASM` respectively and colours the VGA display using a colour gradient and prints a "Hello World!" text by calling `VGA_draw_point_ASM` and `VGA_write_char_ASM`. The result of this test shown in Figure 1 is as expected.



Figure 1: VGA driver test output

2 Reading Keyboard Input

2.1 PS/2 Driver

To detect and read keystroke events through the PS/2 keyboard protocol, we create a PS/2 driver that inputs a memory address to store the data read from the PS/2 keyboard port if unaccessed data is available in the FIFO that stores data received from the PS/2 device. We detect the availability of unread data by checking the 15th (`RVALID`) bit in the PS/2 data register. The subroutine returns the value of the `RVALID` bit.

This subroutine named `read_PS2_data_ASM` isolates the `RVALID` bit by performing an `ANDS` operation between the content of the PS/2 data register and 0x8000 (number only containing 1 as the 15th bit). If the `RVALID` bit is equal to 0, we store 0 in the return register `R0` and branch back to the caller. Otherwise, we read the first 8 bits of the PS/2 data register using a byte load operation (`LDRB`) and store the loaded data into the address passed in the first argument `R0` using `STRB`. Finally, 1 is stored into `R0` before branching to the caller.

2.2 Testing the PS/2 Driver

To test the implemented PS/2 driver, we use the provided program that displays the make and break codes corresponding using the PS/2 keyboard device. We run the program and input the characters "a", "b", and "c" in sequence. The VGA display after performing this test is presented in Figure 2. As shown in Figure 2, 9 codes were printed on the VGA display. We can verify that the first, second, and third triplets correspond to characters "a", "b", and "c" respectively. The first code in each triplet corresponds to the make code while the seconds and third codes together correspond to the break code. For instance, the first displayed triplet (`1C F0 1C`) corresponds to character "a" with `1C` being the make code and `F0 1C` being the break code.

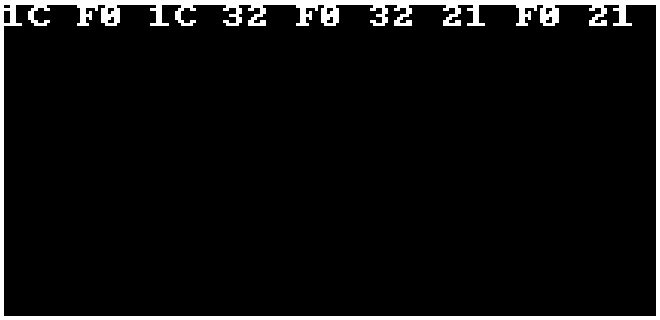


Figure 2: PS/2 driver test output

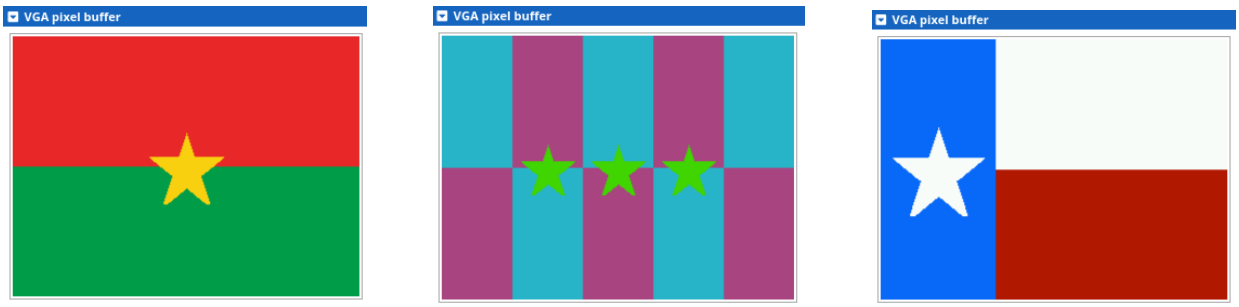
3 Putting Everything Together: Vexillology

In this last part, we use all implemented subroutines and a provided starter code to complete a program that uses keys "A" and "D" received through the PS/2 keyboard interface to cycle through flags on the VGA display. We are provided with two subroutines, `draw_rectangle` and `draw_star` to implement two subroutines named `draw_real_life_flag` and `draw_imaginary_flag_` that draw real-life and imaginary flags as the names suggest.

For the real-life flag, we implement `draw_real_life_flag` to draw the flag of Burkina Faso. This flag is composed of two horizontal red and green bands with a yellow five-pointed star. The two horizontal bands are drawn using the `draw_rectangle` subroutine. The first call to `draw_rectangle`, draws the red band on the top half of the screen. The subroutine is called by passing 0 and 0 for the starting coordinates x and y, 320 for the width and 120 ($^{240}/_{2}$) for the height of the rectangle. The mentioned arguments are passed in `R0` to `R3` respectively. The colour red is pushed on to the stack and popped off the stack once the procedure call is finished. The second call to `draw_rectangle` draws the green band occupying the bottom half of the flag. We pass the same width, height, and x argument as the first call but update the y argument to 120 and the last argument pushed to the stack to the colour green. Finally, the yellow star is drawn by calling the `draw_star` subroutine, by passing the coordinates of the centre of the screen ($x = {}^{340}/_{2} = 160$, $y = {}^{240}/_{2} = 120$), a radius to 35 and the colour yellow. The resulting flag is displayed in Figure 3 (a).

We use the same two helper subroutines with the described argument to implement the `draw_imaginary_flag` subroutine to display a slightly more complex flag on the VGA display illustrated in Figure 3 (b).

We test all subroutines we implemented in this lab using the provided program. Cycling through the flags using the PS/2 keyboard, flags in Figure 3 are displayed. We can therefore deduce the correctness of all our implementations.



(a) Real-life flag (flag of Burkina Faso)

(b) Imaginary flag

(c) flag of Texas

Figure 3

4 Conclusion

To conclude, in this lab, we implemented drivers for the VGA display and PS/2 keyboard port of the DE1-SoC. We exercised our knowledge of ARM assembly and input/output and used multiple sources of documentation. We explored ways of testing our subroutines as we implement them and practiced concepts such as code reusability and defensive programming. We finally integrated all the software components written into a more complex application requiring perfectly functioning components.